

ServoraBd

Local Service Marketplace

PRODUCT REQUIREMENTS DOCUMENT

A marketplace connecting customers with verified local service providers — built around a transparent, multi-factor trust model rather than a single star rating.

Version	1.1
Status	Draft for Build
Date	25 September 2026
Author	Jahid H. R.
Stack	Django · DRF · React (JS) · PostgreSQL

Contents

1	Executive Summary
2	Problem Statement
3	Users, Roles and Permissions
4	Scope and Release Plan
5	Functional Requirements
6	Data Model
7	The Trust Score Engine
8	System Architecture
9	API Specification
10	Frontend Specification
11	Environment and Deployment
12	Non-Functional Requirements
13	Success Metrics
14	Delivery Plan
15	Open Questions

1. Executive Summary

ServoraBd is a two-sided marketplace that connects customers who need home and device services with verified local service providers. The first market is urban Bangladesh, starting with Dhaka.

Today, finding a trustworthy electrician, plumber, AC technician or appliance repairman is an informal process. Customers ask neighbours, relatives, or post in Facebook groups. The result is unpredictable: prices are negotiated blind, quality is unknown until the worker arrives, and there is no recourse when a job goes wrong. Providers suffer the mirror problem — a genuinely skilled technician has no portable, credible way to prove they are skilled, so they compete on price against unqualified workers.

ServoraBd solves both halves with a single mechanism: a **transparent, multi-factor Trust Score**. Rather than reducing a provider to one star rating that is easy to fake and impossible to interpret, ServoraBd computes trust from measurable behavioural signals — completed jobs, completion rate, cancellation rate, median response time, review quality, and verification depth — and shows the customer exactly how the number was reached.

Product thesis

In a market with no institutional trust infrastructure, the marketplace that manufactures credible trust wins — not the one with the most listings. Trust is the product; the booking flow is the delivery mechanism.

1.1 Goals

- 1 Let a customer go from 'my AC is broken' to a confirmed booking with a verified provider in under three minutes.
- 2 Give every provider a trust profile that is earned through behaviour, explainable to customers, and resistant to manipulation.
- 3 Give providers a dependable stream of work and a transparent view of their earnings and standing.
- 4 Operate the platform with enough audit trail that any trust score, booking state, or payout can be reconstructed after the fact.

1.2 Non-Goals for v1

- Not a real-time dispatch system. Bookings are scheduled, not instantly hailed like a ride-share.
- Not an employment platform. Providers are independent contractors; ServoraBd does not manage employment, insurance, or taxation.
- Not a product marketplace. Parts and materials are handled off-platform between customer and provider in v1.
- Not a multi-city or multi-country platform at launch. Geography is modelled generically but seeded for Dhaka only.

2. Problem Statement

2.1 The Customer's Problem

A customer with a broken appliance faces a sequence of frictions, each of which loses candidates:

Friction	What actually happens	Cost to customer
Discovery	Asks neighbours or posts in a Facebook group; waits hours for replies from strangers.	Hours of delay; no filtering.
Credibility	Has no way to tell a 15-year electrician from someone who watched a video.	Risk of damage, re-work, or injury.
Price	Quoted verbally on arrival, after the worker has already travelled. Hard to refuse.	Systematic overcharging.
Reliability	Worker agrees to come at 10am and arrives at 4pm, or not at all.	Lost work day.
Recourse	If the repair fails a week later, there is no record the transaction happened.	Total loss.

2.2 The Provider's Problem

The skilled provider's problem is the inverse and is the reason supply quality stays low:

- **No portable reputation.** Twelve years of satisfied customers produce no asset the provider can show a thirteenth customer.
- **Price competition against the unqualified.** Because quality is invisible before purchase, customers rationally choose the cheapest quote — a market for lemons.
- **Unpredictable demand.** Income depends on word of mouth, which is seasonal and geographically narrow.
- **Wasted travel.** Providers travel to quote jobs they do not win, and that unpaid travel is priced into the jobs they do win.

2.3 Why a Plain Star Rating Does Not Fix This

The obvious solution — let customers leave a 1–5 star rating — fails in predictable ways, and avoiding those failures is the core technical premise of this product:

Failure mode	Mechanism
Rating compression	Nearly all ratings cluster at 4.5–5.0, so the score carries almost no information. A 4.7 and a 4.9 are statistically indistinguishable but look different.
Small-sample noise	A provider with one 5-star review outranks a provider with two hundred jobs averaging 4.8. The first number is not a measurement; it is an accident.
No behavioural coverage	A rating captures only the jobs that were completed. A provider who accepts ten bookings and cancels eight can hold a perfect rating on the two they finished.
Gameable	Ratings are cheap to manufacture through friends and fake accounts, because leaving one costs nothing and requires no verified transaction.
Opaque	The customer cannot tell whether 4.7 means 'usually good' or 'excellent but occasionally disastrous'. Two very different risk profiles collapse into one number.

ServoraBd's Trust Score is designed specifically against these five failure modes. Section 7 specifies it in full.

3. Users, Roles and Permissions

3.1 Personas

Rumana — the Customer

34, works in a bank in Dhanmondi, lives in a flat with her family. Her AC stopped cooling on a Thursday evening. She needs someone reliable before the weekend, is willing to pay a fair rate, and is far more worried about being cheated or having her time wasted than about saving 300 taka. She will not use a platform that takes more than a few minutes to learn.

Kamal — the Service Provider

41, AC and refrigeration technician with fourteen years of experience, works across Mohammadpur and Dhanmondi. He owns a smartphone and uses it constantly but does not read English fluently. He wants a steady flow of jobs near where he already is, clear expectations before he travels, and prompt payment. He will abandon any app that makes him do data entry.

Tanvir — the Admin / Operations Officer

Reviews provider verification submissions, investigates disputes, monitors trust-score anomalies, and manages the service catalogue. Needs speed and a full audit trail more than visual polish.

3.2 Role Capability Matrix

Capability	Guest	Customer	Provider	Admin
Browse categories and services	Yes	Yes	Yes	Yes
Search and view provider profiles	Yes	Yes	Yes	Yes
View a provider's trust breakdown	Yes	Yes	Yes	Yes
Create a service request	No	Yes	No	No
Accept / decline a request	No	No	Yes	No
Mark a job complete	No	Confirm	Propose	Override
Leave a review	No	Yes	Reply only	Moderate
Manage own availability & pricing	No	No	Yes	No
View own earnings ledger	No	No	Yes	Yes
Approve / reject verification	No	No	No	Yes
Suspend an account	No	No	No	Yes
Recompute / inspect trust audit	No	No	Own only	Yes

Account model. One User record holds authentication. A user may hold a CustomerProfile, a ProviderProfile, or both — a plumber is also someone who needs their laptop fixed. The active role is a context switch in the UI, not a separate account, and the JWT carries the set of roles the user actually holds.

4. Scope and Release Plan

Scope is phased so that Phase 1 is independently shippable and demonstrates the product thesis. Everything that is not required to prove 'trust drives booking' is deferred.

4.1 Phase 1 — MVP (in scope)

Area	Included in Phase 1
Accounts	Email/phone registration, OTP phone verification, JWT auth, password reset, role switching.
Catalogue	Category and service taxonomy, admin-managed, seeded with the eight launch categories.
Providers	Profile, offered services with per-service pricing, service areas, weekly availability, document upload for verification.
Discovery	Browse by category, filter by area, price band, trust tier and availability; sort by trust, price or distance.
Booking	Full request lifecycle from request through accept, scheduled, in-progress, completion and confirmation.
Reviews	Post-completion review with multi-dimension ratings, verified-transaction gating, one provider reply.
Trust	Full Trust Score engine, factor breakdown UI, audit snapshots, admin inspection.
Payments	Cash-on-completion recorded on-platform; commission accrued to a provider earnings ledger.
Admin	Verification review queue, catalogue management, account suspension, trust audit viewer.
Notifications	Transactional email plus in-app notification centre.

4.2 Phase 2 — Post-MVP

- Online payment via bKash, Nagad and card through SSLCommerz; escrow hold and automated provider payout.
- Dispute and refund workflow with admin adjudication.
- Abuse reporting (Report model) for reviews, profiles and messages.
- In-app customer-provider messaging.
- SMS and push notifications.
- Bangla language interface (full bilingual support).
- Provider-side quote negotiation for non-fixed-price jobs.

4.3 Phase 3 — Scale

- Multi-city expansion with per-city catalogue and pricing.
- Provider teams / small firms with multiple technicians under one profile.
- Recurring bookings (monthly AC servicing, weekly cleaning).
- Subscription tier for providers (priority placement, lower commission).
- Demand-based dynamic pricing guidance for providers.
- Public trust API for third-party verification.

4.4 Explicitly Descoped

The following were considered and deliberately excluded. Recording the decision prevents scope drift:

Item	Reason for exclusion
Real-time GPS tracking of providers	High implementation and privacy cost; scheduled bookings do not need it. Revisit only if same-day urgent becomes a major segment.
In-app parts / materials commerce	A second marketplace with its own logistics and returns problem. Out of scope entirely.
Provider background checks by third party	No reliable vendor at an acceptable price in this market. NID verification plus behavioural trust is the v1 substitute.
Customer-side subscriptions	No evidence of willingness to pay before the marketplace has liquidity.

5. Functional Requirements

Requirements are identified as FR-x.y so they can be referenced from tickets and tests. Priority: **M** = must have for Phase 1, **S** = should have, **C** = could have.

5.1 Accounts and Identity

ID	Requirement	Pri
FR-1.1	A visitor can register as a customer or provider using a Bangladeshi mobile number (+8801XXXXXXXX) or an email address.	M
FR-1.2	Phone numbers are verified by a 6-digit OTP valid for 5 minutes, rate-limited to 3 sends per number per hour.	M
FR-1.3	Authentication issues a short-lived JWT access token (15 min) and a rotating refresh token (14 days).	M
FR-1.4	A user holding both profiles can switch active role without re-authenticating.	M
FR-1.5	A provider submits NID front/back images and an optional trade certificate for admin identity verification.	M
FR-1.6	Verification documents are stored private (never publicly addressable) and are purged 90 days after a decision.	M
FR-1.7	A user can request deletion of their account; personal data is anonymised while transaction records are retained.	S

5.2 Service Catalogue

ID	Requirement	Pri
FR-2.1	Admin manages a two-level taxonomy: ServiceCategory (e.g. Electrical) containing Service (e.g. Ceiling fan installation).	M
FR-2.2	Each Service defines a pricing model: fixed, hourly, or visit-fee-plus-quote.	M
FR-2.3	Each Service carries a platform-suggested price band used to flag provider prices as unusually high or low.	S
FR-2.4	Launch catalogue covers eight categories: Electrical, Plumbing, AC & Refrigeration, Computer & IT, Painting, Cleaning, Appliance Repair, Car Mechanic.	M

5.3 Provider Profile and Offerings

ID	Requirement	Pri
FR-3.1	A provider builds a profile with display name, photo, bio, years of experience and spoken languages.	M
FR-3.2	A provider attaches Services they offer, each with their own price and estimated duration (ProviderService).	M
FR-3.3	A provider defines one or more service areas by selecting Locations (thana / area level).	M
FR-3.4	A provider defines recurring weekly availability as day-of-week time windows, plus dated exceptions for leave.	M
FR-3.5	A provider can set an instant on/off 'accepting work' toggle that hides them from search without deleting anything.	M

ID	Requirement	Pri
FR-3.6	A provider profile displays the full trust breakdown, not a single score, to any viewer.	M
FR-3.7	A provider can upload up to 10 photographs of past work.	C

5.4 Discovery and Search

ID	Requirement	Pri
FR-4.1	A customer can browse providers by category and by location, with both required before results are shown.	M
FR-4.2	Results can be filtered by price band, trust tier, verification status and availability on a chosen date.	M
FR-4.3	Results can be sorted by trust score, price ascending, or proximity. Default sort is trust.	M
FR-4.4	Search results are paginated at 20 per page and respond in under 400 ms at p95.	M
FR-4.5	A customer can compare up to three providers side by side on price, trust factors and availability.	S
FR-4.6	Providers who are unverified are shown but visually marked and never rank above a verified provider at equal trust.	M

5.5 Booking Lifecycle

ID	Requirement	Pri
FR-5.1	A customer creates a ServiceRequest specifying service, address, preferred time window, and a free-text problem description with optional photos.	M
FR-5.2	A request may be directed at one provider or broadcast to all matching providers in the area.	M
FR-5.3	A provider is notified of a request and may accept or decline with a reason. First acceptance on a broadcast request wins; others are auto-closed.	M
FR-5.4	Acceptance creates a Booking with a confirmed scheduled datetime and locks the agreed price.	M
FR-5.5	An unanswered request expires after 24 hours and is recorded as expired — not as a provider decline.	M
FR-5.6	Either party may cancel before the scheduled time. The canceller, reason and timestamp are recorded.	M
FR-5.7	A provider marks a job in-progress on arrival and completed when finished; the booking then awaits customer confirmation.	M
FR-5.8	If the customer does not confirm within 72 hours, the booking auto-confirms and becomes reviewable.	M
FR-5.9	Every state transition is written to an immutable BookingEvent log with actor, timestamp and reason.	M

5.6 Reviews

ID	Requirement	Pri
FR-6.1	Only the customer on a confirmed-complete booking may review it, and only once. There is no free-floating review.	M
FR-6.2	A review carries an overall 1–5 rating plus sub-ratings for punctuality, quality, professionalism and price fairness.	M

ID	Requirement	Pri
FR-6.3	Reviews may be submitted up to 30 days after confirmation, after which the window closes.	M
FR-6.4	A provider may post exactly one public reply per review.	S
FR-6.5	A review may not be edited after 24 hours; the edit history is retained internally.	S
FR-6.6	Admin may hide a review, with a recorded reason; hidden reviews are excluded from trust computation.	M

5.7 Payments and Earnings (Phase 1 scope)

ID	Requirement	Pri
FR-7.1	Phase 1 settles in cash on completion. The provider records the collected amount; the customer confirms it.	M
FR-7.2	A platform commission (default 12%) is accrued as a payable on the provider's ledger at confirmation.	M
FR-7.3	A provider sees an earnings dashboard: gross, commission, net, and outstanding payable, by period.	M
FR-7.4	A mismatch between provider-recorded and customer-confirmed amounts flags the booking for admin review.	S
FR-7.5	The Payment model is designed for gateway integration in Phase 2 without schema migration of Booking.	M

5.8 Notifications

ID	Requirement	Pri
FR-8.1	Transactional email is sent for: request received, request accepted/declined, booking reminder (24h and 2h before), completion, review invitation, verification decision.	M
FR-8.2	An in-app notification centre lists all events with read/unread state.	M
FR-8.3	Notification dispatch failures never fail the user's request; they are logged and retried by a sweep command.	M
FR-8.4	Notification templates are database-backed so copy can be changed without deployment.	C

5.9 Scheduled Job Operability

Phase 1 runs its recurring work as scheduled management commands (section 8.4). Because nothing in the request cycle would notice a scheduler that has stopped, observability is a functional requirement rather than an operational nicety.

ID	Requirement	Pri
FR-9.1	Every scheduled command writes a JobRun record: job name, started/finished timestamps, rows affected, and outcome.	M
FR-9.2	An admin panel lists each job with its last-successful-run time and highlights any job overdue by more than twice its interval.	M
FR-9.3	Every scheduled command is idempotent — a repeated or overlapping run must not double-apply any effect.	M

ID	Requirement	Pri
FR-9.4	A command that raises exits non-zero and records the failure, so the scheduler and the panel both surface it.	M
FR-9.5	A run_scheduled_jobs command invokes every sweep once in dependency order, for manual testing and recovery.	S

6. Data Model

PostgreSQL is the chosen database. The decision is driven by three concrete needs, each of which is materially harder on MySQL: **JSONB** with GIN indexing for trust-score factor snapshots, **PostGIS** or **cube/earthdistance** for service-area and proximity matching, and native **full-text search** with SearchVector for provider and service search. Partial and expression indexes are also used heavily in the booking tables.

6.1 Entity Overview

Entity	Purpose	Key relationships
User	Authentication and identity. Custom user model with phone as an alternate identifier.	1:0..1 CustomerProfile, 1:0..1 ProviderProfile
CustomerProfile	Customer-side data: saved addresses, default area, booking history.	FK User
ProviderProfile	Provider-side data: bio, experience, verification state, denormalised trust fields.	FK User
ServiceCategory	Top-level taxonomy (Electrical, Plumbing...).	1:N Service
Service	A concrete service with a pricing model and suggested price band.	FK ServiceCategory
ProviderService	Join: which provider offers which service, at what price and duration.	FK ProviderProfile, FK Service
Location	Hierarchical geography: city / thana / area, with a centroid point.	Self-FK parent; M:N ProviderProfile
ServiceArea	Join: a provider serves a location, optionally with a travel surcharge.	FK ProviderProfile, FK Location
Availability	Recurring weekly window, plus dated exceptions.	FK ProviderProfile
ServiceRequest	A customer's demand before a provider commits.	FK CustomerProfile, FK Service, FK Location
Booking	A committed job with locked price and schedule.	1:1 ServiceRequest, FK ProviderProfile
BookingEvent	Append-only state-transition log.	FK Booking
Review	Customer's post-completion assessment.	1:1 Booking
Payment	Money movement record, cash or gateway.	FK Booking
TrustSnapshot	Immutable record of a computed trust score and its factor inputs.	FK ProviderProfile
VerificationDocument	Submitted NID / certificate and its review decision.	FK ProviderProfile
Notification	In-app notification item.	FK User
JobRun	Execution record for one scheduled management command; makes a stalled scheduler visible.	—

6.2 Key Field Definitions

ProviderProfile carries denormalised trust fields so that search does not need to recompute or join on every query:

```
class ProviderProfile(models.Model):
    user = OneToOneField(User, on_delete=CASCADE)
    display_name = CharField(max_length=120)
    bio = TextField(blank=True)
    experience_years = PositiveSmallIntegerField(default=0)
    is_accepting_work = BooleanField(default=True)

    identity_verified = BooleanField(default=False)
    phone_verified = BooleanField(default=False)
    skill_verified = BooleanField(default=False)

    # denormalised trust cache - recomputed by engine, never hand-set
    trust_score = DecimalField(max_digits=5, decimal_places=2,
                              default=0) # 0-100
    trust_tier = CharField(max_length=20, default='new')
    trust_computed_at = DateTimeField(null=True)

    jobs_completed = PositiveIntegerField(default=0)
    jobs_cancelled = PositiveIntegerField(default=0)
    jobs_accepted = PositiveIntegerField(default=0)
    requests_received = PositiveIntegerField(default=0)
    median_response_s = PositiveIntegerField(null=True)

    class Meta:
        indexes = [
            Index(fields=['-trust_score']),
            Index(fields=['is_accepting_work', '-trust_score']),
        ]
```

Booking is the transactional core. The state machine is enforced in the service layer, never by direct field assignment in a view:

```
class Booking(models.Model):
    class State(models.TextChoices):
        PENDING = 'pending'
        ACCEPTED = 'accepted'
        SCHEDULED = 'scheduled'
        IN_PROGRESS = 'in_progress'
        AWAITING_CONFIRM = 'awaiting_confirm'
        COMPLETED = 'completed'
        CANCELLED_CUST = 'cancelled_customer'
        CANCELLED_PROV = 'cancelled_provider'
        EXPIRED = 'expired'
        DISPUTED = 'disputed'

    request = OneToOneField(ServiceRequest, on_delete=PROTECT)
    provider = ForeignKey(ProviderProfile, on_delete=PROTECT)
    state = CharField(choices=State.choices,
                     default=State.PENDING)

    scheduled_for = DateTimeField()
    agreed_price = DecimalField(max_digits=10, decimal_places=2)
    final_price = DecimalField(null=True, ...)
    accepted_at = DateTimeField(null=True)
    completed_at = DateTimeField(null=True)
    confirmed_at = DateTimeField(null=True)
    cancelled_by = CharField(null=True) # customer|provider|system
    cancel_reason = TextField(blank=True)
```

TrustSnapshot is what makes the score auditable. Every recomputation writes a new row; rows are never updated or deleted. `factors` is JSONB holding every input and sub-score, so any historical score can be fully explained:

```
class TrustSnapshot(models.Model):
    provider      = ForeignKey(ProviderProfile, related_name='snapshots')
    score         = DecimalField(max_digits=5, decimal_places=2)
    tier           = CharField(max_length=20)
    factors       = JSONField()      # full input + sub-score record
    algo_version  = CharField(max_length=10)  # e.g. 'v1.0'
    trigger       = CharField(max_length=40)  # booking_completed, ...
    created_at    = DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [Index(fields=['provider', '-created_at'])]
        # append-only: enforced in service layer + DB trigger
```

6.3 Integrity Rules

- A Review requires a Booking in state COMPLETED. Enforced by a CheckConstraint plus a service-layer guard.
- Booking.agreed_price is immutable after acceptance; a change requires cancellation and re-booking.
- ServiceRequest and Booking use on_delete=PROTECT — transaction history is never destroyed by a profile deletion.
- Account deletion anonymises User fields and detaches the profile, leaving bookings and their aggregate trust contribution intact.
- All money is DecimalField. No floats anywhere in the payment or earnings path.

7. The Trust Score Engine

This is the product's differentiating feature and the most technically substantial part of the build. It is specified here in full detail: the factors, their formulas, the composition, the anti-gaming design, the recomputation triggers, and how it is presented to the customer.

Design principles

1. Every factor must be derived from a recorded platform event, never self-reported. 2. The score must be explainable in one screen to a non-technical customer. 3. A new provider must be able to reach a good score through good behaviour within a realistic number of jobs. 4. No single factor may dominate, so that gaming one signal is not enough.

7.1 The Six Factors

#	Factor	Signal it captures	Weight
F1	Verification Depth	Has this person proven who they are?	20%
F2	Job Volume	Is there enough evidence to judge them at all?	15%
F3	Completion Reliability	When they commit, do they finish?	20%
F4	Cancellation Discipline	How often do they break a commitment?	15%
F5	Responsiveness	How long does a customer wait for an answer?	10%
F6	Review Quality	What do verified customers actually say?	20%

Each factor produces a sub-score in $[0, 100]$. The composite is the weighted sum, then adjusted by penalties. Weights are stored in configuration, not hard-coded, and the `algo_version` on each snapshot records which set produced a given score.

F1 — Verification Depth (20%)

A step function over discrete, admin-confirmed facts. This is the one factor a provider can maximise immediately, which is intentional: it gives a brand-new provider a way to signal seriousness on day one.

```
F1 = 0
F1 += 30 if phone_verified      (OTP confirmed)
F1 += 40 if identity_verified  (NID reviewed by admin)
F1 += 20 if skill_verified     (trade certificate accepted)
F1 += 10 if address_verified   (service address confirmed)
# capped at 100
```

F2 — Job Volume (15%)

Volume should produce strongly diminishing returns: the difference between 0 and 10 jobs is enormous; between 200 and 210 it is meaningless. A logarithmic curve saturating at 100 jobs captures this.

```
F2 = 100 * ln(1 + completed) / ln(1 + 100)

# completed=0    -> 0.0
# completed=5    -> 38.9
# completed=25   -> 70.6
# completed=100  -> 100.0 (saturated)
# completed=300  -> capped at 100
```


F3 — Completion Reliability (20%)

The naive form is completed / accepted. That is unusable at low volume — one provider with 1/1 would score 100 and outrank a provider with 189/200. The fix is **Bayesian smoothing**: blend the observed rate toward a prior platform average, with the prior's influence decaying as real evidence accumulates.

```
mu    = 0.90      # platform prior completion rate
k     = 10        # prior strength, in pseudo-jobs

F3 = 100 * (completed + k * mu) / (accepted + k)

# 1 of 1 completed    -> 100*(1 + 9)/(1 + 10) = 90.9
# 0 of 1 completed    -> 100*(0 + 9)/(1 + 10) = 81.8
# 189 of 200          -> 100*(189 + 9)/(200 + 10) = 94.3
# 5 of 20 completed   -> 100*(5 + 9)/(20 + 10) = 46.7
```

Note the behaviour that makes this correct: the 1-of-1 provider scores 90.9, *below* the 189-of-200 veteran at 94.3. A single good outcome no longer beats a long record. The prior strength $k = 10$ means it takes roughly ten real jobs before a provider's own history outweighs the platform assumption.

F4 — Cancellation Discipline (15%)

Cancellations are weighted by how much damage they cause. Cancelling two days ahead is a minor inconvenience; cancelling an hour before, after the customer has taken time off work, is severe. Recent cancellations also matter more than old ones.

```
# per-cancellation damage weight by notice given
w = 1.0  if notice >= 24h
w = 2.0  if 4h <= notice < 24h
w = 4.0  if notice < 4h
w = 6.0  if no-show (never arrived, never cancelled)

# time decay: a cancellation 'ages out' with a 180-day half-life
decay(d) = 0.5 ** (days_ago / 180)

weighted_cancels = SUM over cancellations of ( w * decay(d) )
cancel_rate      = weighted_cancels / max(accepted, 1)

F4 = 100 * (1 - min(cancel_rate, 1.0)) ** 1.5
```

The exponent of 1.5 makes the penalty superlinear: a 10% weighted cancellation rate costs about 15 points, but a 30% rate costs about 41. Mild unreliability is tolerated; chronic unreliability is not.

F5 — Responsiveness (10%)

Measured as the median — not mean — seconds from request notification to the provider's accept or decline, over the last 30 responses. Median resists the distortion of one holiday-week outlier. Requests that expire unanswered are counted at the ceiling value, so ignoring requests is not a way to protect the metric.

```
t = median_response_seconds (expired requests counted as 24h)

F5 = 100                                if t <= 5 min
F5 = 100 - 30*(t-5)/55                  if 5 min < t <= 60 min
F5 = 70 - 30*(t-60)/180                  if 60 min < t <= 4 h
F5 = 40 - 40*(t-240)/1200                if 4 h < t <= 24 h
F5 = 0                                  if t > 24 h

# providers with fewer than 5 responses: F5 = 60 (neutral)
```

F6 — Review Quality (20%)

Two corrections to a raw star average. First, the same Bayesian smoothing applied in F3, so a single 5-star review does not outrank a long record. Second, **recency weighting** — a provider's performance last month is more predictive than their performance two years ago.

```
# recency weight, 365-day half-life
rw(d) = 0.5 ** (days_ago / 365)

weighted_sum = SUM( rating_i * rw(d_i) )
weighted_count = SUM( rw(d_i) )

mu_r = 4.2      # platform prior mean rating
k_r = 5         # prior strength, in pseudo-reviews

bayes = (weighted_sum + k_r * mu_r) / (weighted_count + k_r)
F6 = 100 * (bayes - 1) / 4      # map 1-5 star onto 0-100
```

7.2 Composition and Penalties

```
base = 0.20*F1 + 0.15*F2 + 0.20*F3 + 0.15*F4
      + 0.10*F5 + 0.20*F6

# --- hard penalties, applied after the weighted sum ---
penalty = 0
penalty += 15  if an upheld dispute in the last 90 days
penalty += 25  if two or more upheld disputes in the last 180 days
penalty += 10  if a confirmed no-show in the last 30 days
penalty += 20  if currently under admin investigation

TRUST = clamp(base - penalty, 0, 100)
```

Penalties are applied after weighting rather than inside a factor so that a serious incident cannot be diluted by strong performance elsewhere — which is exactly the failure a pure weighted average would produce.

7.3 Trust Tiers

The numeric score is used for ranking; the tier is what the customer reads. Tiers deliberately do *not* use star imagery, to avoid being mentally collapsed back into a rating.

Tier	Score	Additional requirement	Meaning to customer
Trusted Pro	85–100	≥25 jobs, identity verified	Long, consistently strong record.
Established	70–84	≥10 jobs, identity verified	Proven, reliable, well reviewed.
Rising	55–69	≥3 jobs	Good early record, still building history.
New	any	<3 completed jobs	Not yet enough evidence. Shown honestly as new.
Under Review	any	Active investigation or suspension	Flagged; booking is blocked.

The job-count gates are deliberate and independent of the score. A provider may hold a high score and still sit in a lower tier because there is not yet enough evidence to promote them — see the worked example in 7.4, where a score of 84.2 is still only 'Rising'. Score drives ranking; tier communicates confidence. Collapsing the two would reintroduce exactly the small-sample problem described in 2.3.

A provider below 40 with 10 or more jobs is automatically queued for admin review rather than silently buried in search results.

7.4 Worked Examples

Both examples below are reproduced exactly by the reference implementation in `docs/verify_trust_math.py`, which asserts every figure in these tables. That script is the specification of record for the arithmetic and the starting point for `apps/trust/factors.py`.

Kamal — experienced, newly joined. Fourteen years of off-platform experience, but only 4 jobs on ServoraBd. Fully verified including trade certificate. 4 of 4 completed, no cancellations, median response 8 minutes, 4 reviews averaging 4.9.

Factor	Input	Sub-score	Weighted
F1 Verification	phone+NID+skill = 90	90.0	18.00
F2 Volume	4 completed	34.9	5.23
F3 Completion	4 of 4, k=10	92.9	18.57
F4 Cancellation	0 cancels	100.0	15.00
F5 Response	median 8 min	98.4	9.84
F6 Reviews	4 @ 4.9, k=5	87.8	17.56
Base			84.19
Penalties	none		0
TRUST			84.2 — Rising

This is the desired outcome, and the interaction between score and tier is the subtle part. Kamal's score is a strong 84.2 — verification and flawless early behaviour are rewarded immediately, so he ranks highly in search from his first week. But his *tier* is 'Rising', not 'Established', because the tier gate requires ten completed jobs and he has four. The score says 'everything we can measure about this person is excellent'; the tier says 'we have not measured very much yet'. Both statements are true and the customer should see both. He is not punished for being new, and he is not over-credited either — six more good jobs promote him to 'Established' with no change in behaviour.

Shakib — high rating, poor reliability. The provider a plain star rating would hide. 40 requests received, 30 accepted, 18 completed, 12 cancelled (3 of them at under 4 hours' notice, weighted 4×; the other 9 with a day or more, weighted 1× — a weighted rate of $21/30 = 0.70$). Phone verified only. Median response 6 hours. 18 reviews averaging 4.8.

Factor	Input	Sub-score	Weighted
F1 Verification	phone only = 30	30.0	6.00
F2 Volume	18 completed	63.8	9.57
F3 Completion	18 of 30, k=10	67.5	13.50
F4 Cancellation	wtd rate 0.70	16.4	2.46
F5 Response	median 6 h	36.0	3.60
F6 Reviews	18 @ 4.8, k=5	91.7	18.35
Base			53.48
Penalties	none		0
TRUST			53.5 — Rising

This example is the entire argument for the product. Shakib's star rating is **4.8** — on any conventional marketplace he sits just behind Kamal's 4.9 and looks excellent. In reality he abandons 40% of the jobs he accepts, a quarter of those at the last minute, and takes six hours to reply. ServoraBd surfaces that: 53.5, 'Rising', with a visible cancellation figure. The customer sees the risk *before* booking, not after. Note also that his review score (F6 = 91.7) is the *highest* of his six factors — the exact signal a star-rating marketplace would show, and the one most misleading here.

7.5 Anti-Gaming Design

Attack	Countermeasure
Fake reviews from friends	Reviews require a completed booking with a recorded payment. Manufacturing one costs a real transaction and a real commission.
Self-booking with a second account	Duplicate detection on device fingerprint, phone, address and payment instrument. Customer-provider pairs with anomalous repeat frequency are flagged for admin review.
Declining hard jobs to protect completion rate	Declines are recorded and feed F5; a decline rate above 60% triggers review. Completion rate is measured against <i>accepted</i> , so refusing work still costs responsiveness and volume.
Cancelling as 'customer requested' to dodge F4	Customer-initiated cancellations must be confirmed by the customer, who is asked independently. Unconfirmed claims count against the provider.
Review extortion ('give me 5 stars or I won't finish')	Reviews are double-blind: neither party sees the other's review until both are submitted or the 14-day window closes.
Volume farming with trivial cheap jobs	F2 is logarithmic and saturates at 100 jobs, so farming has sharply diminishing value. Jobs below a minimum value contribute at reduced weight.
Abandoning a damaged account and re-registering	NID is unique across provider accounts. A new profile on a previously penalised NID inherits the prior record.

7.6 Computation Strategy

Trust is recomputed by the trust engine on the triggers below. In Phase 1 the event-driven triggers run synchronously at the end of the originating transaction, and the nightly sweep runs as a management command; section 8.4 covers the mechanism and 8.5 the later move to a task queue. The trigger list itself does not change either way:

- Booking reaches COMPLETED or any cancelled state.
- A review is created, edited, or hidden by admin.
- A verification decision is recorded.
- A dispute is opened or resolved.
- Nightly batch for all active providers — required because time-decay terms (F4, F6) change with no new event.

Each run writes a new TrustSnapshot and updates the denormalised cache on ProviderProfile inside one transaction. Because snapshots are append-only, a provider's trust history is chartable, and any past ranking decision can be reconstructed — which is also what makes the score defensible when a provider complains that it dropped.

7.7 Customer-Facing Presentation

The single most important UI rule in this product: **never show the composite score alone**. The breakdown is the feature. The provider card and profile render the factors as concrete facts:

Kamal Hossain - AC & Refrigeration

RISING Trust 84.2 / 100

Identity verified (NID)	[verified]
Phone verified	[verified]
Trade certificate	[verified]

Jobs completed	4
Completion rate	100%
Cancellation rate	0%
Typical response time	8 min
Customer rating	4.9 (4 reviews)

New to ServoraBd - fully verified, strong
early record. [\[How trust is scored\]](#)

The 'How trust is scored' link opens a plain-language explanation of the six factors and their weights. The algorithm is published, not hidden — transparency is what converts a number into trust, and a provider who understands the score can improve it.

8. System Architecture

8.1 Stack

Layer	Technology	Rationale
Frontend	React 18 + Vite (JavaScript)	Component model suits the repeated provider-card and booking-state UI. Plain JavaScript with JSDoc annotations on shared shapes; see 8.6 for the discipline that replaces static typing.
State/data	TanStack Query + Zustand	Server state and UI state kept separate; caching and refetch handled properly.
Styling	Tailwind CSS	Fast iteration; consistent design tokens without a component library's weight.
API	Django 5 + Django REST Framework	Mature ORM for a relational domain; DRF serializers and permissions map cleanly onto the role matrix.
Database	PostgreSQL 18	JSONB for trust snapshots, earthdistance/PostGIS for geo, native full-text search.
Background work	Django management commands + OS scheduler	Phase 1 has no broker. Scheduled sweeps run as commands under Task Scheduler / cron; event-driven work runs synchronously. See 8.5.
Ephemeral state	PostgreSQL with expiry columns	OTP codes and rate-limit counters as rows with expires_at, swept by a management command. No Redis in Phase 1.
Auth	SimpleJWT	Stateless access tokens with refresh rotation and blacklist.
Storage	Local filesystem (dev) / S3-compatible (prod)	Private location for verification documents, public for profile and work photos.
Deploy	VPS: Nginx + Gunicorn + systemd, installed natively	No containers. Proportionate to launch scale and to the operator's current toolset; Docker is an optional later improvement, not a prerequisite.

8.2 Backend Layering

The single most important architectural rule: **business logic lives in a service layer, not in views, serializers or model save() methods**. Booking state transitions and trust computation must be callable from a view, a management command, a test, and later a queue worker, with identical behaviour. This rule is what makes section 8.5's migration to a task queue a one-line change per call site rather than a rewrite.

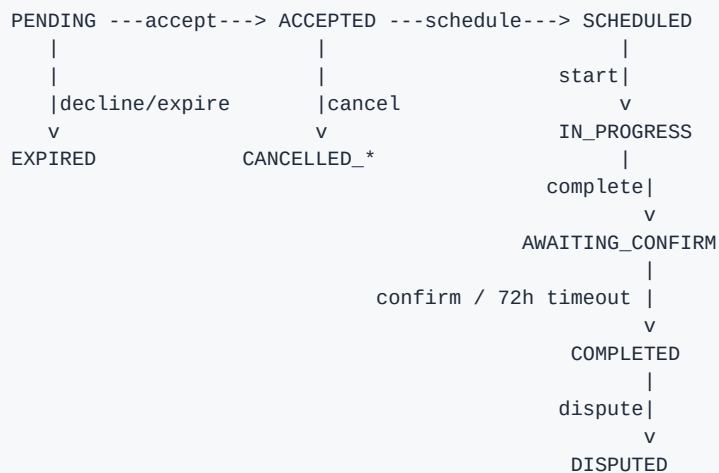
```
apps/
accounts/      User, profiles, OTP, JWT, role switching
catalogue/     ServiceCategory, Service, Location
providers/     ProviderProfile, ProviderService, ServiceArea,
               Availability, verification
bookings/      ServiceRequest, Booking, BookingEvent, state machine
reviews/       Review, moderation
trust/         engine.py, factors.py, TrustSnapshot
payments/      Payment, earnings ledger, commission
notifications/ Notification, templates, dispatch
common/        base models, permissions, pagination, exceptions

# within each app:
models.py      persistence only, no business rules
selectors.py   read queries (kept out of views)
services.py    write operations, transactions, state transitions
serializers.py I/O shape only
views.py       HTTP concerns, permissions, delegation to services
management/commands/ scheduled sweeps -> services

# tasks.py is added per app only when the queue is introduced;
# it will be a thin wrapper that calls the same services.
```

8.3 Booking State Machine

All transitions go through `bookings/services.py`, which validates the transition, writes the `BookingEvent`, and enqueues side effects. Illegal transitions raise a domain exception, never a 500.



8.4 Background Work Without a Task Queue

Phase 1 runs with no message broker and no worker process. Work that would conventionally sit on a queue is split into two kinds, handled differently:

- **Event-driven work** — triggered by a user action, such as recomputing trust after a booking completes. Runs *synchronously* inside the request, at the end of the transaction.
- **Scheduled work** — triggered by the clock, with no user action at all, such as expiring 24-hour-old requests. Runs as a **Django management command** invoked by the operating system's scheduler (Task Scheduler on Windows, cron on the server).

The second kind has no synchronous equivalent — nothing in a request cycle can notice that a request has gone stale — so management commands are a genuine solution here, not a placeholder.

Job	Kind	Phase 1 mechanism	Schedule
recompute_trust	Event	Called synchronously from <code>services.py</code> after a booking, review or verification event.	Immediate
nightly_trust_batch	Scheduled	<code>manage.py recompute_all_trust</code>	03:00 daily
expire_stale_requests	Scheduled	<code>manage.py expire_requests</code>	Every 15 min
auto_confirm_bookings	Scheduled	<code>manage.py auto_confirm</code>	Hourly
send_booking_reminders	Scheduled	<code>manage.py send_reminders</code>	Every 30 min
purge_verification_docs	Scheduled	<code>manage.py purge_documents</code>	Daily
purge_expired_otps	Scheduled	<code>manage.py purge_otps</code> — replaces Redis TTL	Every 10 min
refresh_search_vectors	Event	Updated in the same transaction as the provider record.	Immediate
send_email	Event	Synchronous SMTP. Failures are logged and retried by a sweep command, never surfaced to the user.	Immediate

Every management command must be idempotent. Running `expire_requests` twice in a row must not expire anything twice, because a scheduler will occasionally double-fire or overlap a slow run. Each command therefore selects rows by state and filters on a timestamp, rather than tracking a cursor.

Known limitations, accepted for Phase 1. Synchronous trust recomputation adds latency to the booking-completion response (budgeted at under 2 s, per section 12.1). Synchronous email adds SMTP latency to several endpoints. Both are tolerable at launch volume and both are the reason section 8.5 exists.

8.5 Migration Path to a Task Queue

A task queue — Celery with Redis as its broker — becomes worthwhile when any of these thresholds is crossed. Introducing it earlier adds two processes and a new failure mode for no benefit:

Trigger	Threshold
Booking-completion latency	p95 exceeds 2 s because trust recomputation is running inline.
Provider count	Above roughly 2,000 active providers, the nightly batch stops fitting in its window.
Email volume	SMTP latency becomes visible to users, or a provider outage starts failing requests.
Notification channels	SMS or push is added — these have far worse tail latency than email and cannot run inline.
Scheduling precision	OS-level scheduling proves too coarse or too fragile to operate.

Because all logic already lives in `services.py`, the migration is mechanical. A thin `tasks.py` is added per app, and call sites change by one word:


```
# Phase 1 - synchronous, in bookings/services.py
def confirm_booking(booking_id, actor):
    with transaction.atomic():
        booking = ...
        _write_event(booking, 'confirmed', actor)
        recompute_trust(booking.provider_id)      # inline
        send_review_invite(booking.id)           # inline

# Later - asynchronous. services.py is UNCHANGED except these
# two lines; the functions themselves are not rewritten.
    recompute_trust.delay(booking.provider_id)
    send_review_invite.delay(booking.id)

# trust/tasks.py, added at migration time:
@shared_task(bind=True, max_retries=3)
def recompute_trust(self, provider_id):
    return engine.recompute(provider_id)  # same service call
```

Three rules must hold from day one for that to work, and they cost nothing to follow now:

- 1 **Services take IDs, not model instances.** A queue message must be JSON-serializable; a `Booking` object is not. Passing `booking_id` is also safer synchronously, because it forces a fresh read.
- 2 **Services never touch request.** No worker has one. Anything needed from the request — the acting user, an IP address — is passed as an explicit argument.
- 3 **Services are idempotent.** A queue may deliver the same message twice after a retry. This is the same property the management commands already require, so it is one discipline, not two.

Redis, when it arrives, serves three distinct purposes that are worth separating: it is Celery's *broker* (the message transport), it is a natural home for *ephemeral keyed state* with a TTL such as OTP codes and rate-limit counters, and it can act as a *cache*. Phase 1 solves the second with Postgres rows carrying an `expires_at` column and a sweep command, and does not need the third at all.

8.6 Frontend Without Static Types

The client is plain JavaScript. The risk that TypeScript would otherwise mitigate is concentrated in two places — the booking state machine and the trust factor payload — so the compensating discipline is targeted there rather than applied uniformly:

- **Booking states are a frozen constant object**, never string literals scattered through components. A typo in `BOOKING_STATE.AWAITING_CONFIRM` is an immediate runtime error; a typo in `'awaiting_confirm'` silently renders nothing.
- **API responses pass through a per-resource normaliser** in `src/api/`. Components never read a raw response, so a backend field rename breaks in one known file instead of in six components.
- **JSDoc typedefs** on the shared shapes — provider, booking, trust breakdown. This gives editor autocomplete and inline checking with no build step and no compiler.
- **PropTypes** on the components that carry money, state or trust values. Not on every component — that becomes noise nobody reads.
- **ESLint** with `eslint-plugin-react`, treating unused variables and undefined references as errors.

The single highest-value rule is the normaliser layer. Most defects that static typing would have caught in a project this size are shape mismatches at the API boundary, and a normaliser catches those at one chokepoint regardless of language.

9. API Specification

REST over JSON, versioned at /api/v1/. All list endpoints are paginated and filterable. Errors follow a single envelope so the React client has one error path.

9.1 Conventions

```
Base URL      /api/v1/
Auth          Authorization: Bearer <access_token>
Pagination    ?page=1&page_size=20 (max 100)
Errors        { "error": { "code": "booking_invalid_state",
                          "message": "...",
                          "details": {...} } }
Idempotency   Idempotency-Key header on POST /bookings/ and payments
```

9.2 Endpoint Reference

Authentication and accounts

Method	Endpoint	Purpose	Auth
POST	/auth/register/	Create account (customer or provider)	—
POST	/auth/otp/send/	Send phone OTP	—
POST	/auth/otp/verify/	Verify OTP, mark phone verified	—
POST	/auth/login/	Obtain token pair	—
POST	/auth/refresh/	Rotate access token	—
POST	/auth/logout/	Blacklist refresh token	User
GET/PATCH	/me/	Current user and profiles	User
POST	/me/switch-role/	Change active role context	User

Catalogue and discovery

Method	Endpoint	Purpose	Auth
GET	/categories/	List service categories	Public
GET	/services/	List services, filter by category	Public
GET	/locations/	Location tree for area pickers	Public
GET	/providers/	Search providers (filters below)	Public
GET	/providers/{id}/	Full provider profile	Public
GET	/providers/{id}/trust/	Trust breakdown, all six factors	Public
GET	/providers/{id}/reviews/	Paginated reviews	Public
GET	/providers/{id}/availability/	Free slots for a date range	Public

GET /providers/ filters: service, category, location, min_trust, tier, verified_only, price_min, price_max, available_on, ordering (-trust_score default, price, distance).

Provider management

Method	Endpoint	Purpose	Auth
GET/PATCH	/provider/profile/	Own profile	Provider
GET/POST	/provider/services/	Offered services and pricing	Provider
PATCH/DELETE	/provider/services/{id}/	Update or remove an offering	Provider
GET/PUT	/provider/service-areas/	Areas served	Provider
GET/PUT	/provider/availability/	Weekly windows and exceptions	Provider
POST	/provider/verification/	Upload NID / certificate	Provider
GET	/provider/dashboard/	Summary: pending, upcoming, trust, earnings	Provider
GET	/provider/earnings/	Ledger with period filters	Provider
GET	/provider/trust-history/	Snapshot series for charting	Provider

Requests, bookings and reviews

Method	Endpoint	Purpose	Auth
POST	/requests/	Create a service request	Customer
GET	/requests/	Own requests (role-scoped)	User
GET	/requests/{id}/	Request detail	Owner
POST	/requests/{id}/cancel/	Withdraw a request	Customer
POST	/requests/{id}/accept/	Accept, creating a booking	Provider
POST	/requests/{id}/decline/	Decline with reason	Provider
GET	/bookings/	Own bookings, filter by state	User
GET	/bookings/{id}/	Booking detail with event history	Party
POST	/bookings/{id}/start/	Mark in-progress	Provider
POST	/bookings/{id}/complete/	Mark complete, record amount	Provider
POST	/bookings/{id}/confirm/	Customer confirms completion	Customer
POST	/bookings/{id}/cancel/	Cancel with reason	Party
POST	/reviews/	Create review for a completed booking	Customer
POST	/reviews/{id}/reply/	Provider's single reply	Provider
GET	/notifications/	In-app notification list	User
POST	/notifications/read/	Mark read	User

Admin

Method	Endpoint	Purpose	Auth
GET	/admin/verifications/	Pending verification queue	Admin
POST	/admin/verifications/{id}/decide/	Approve or reject with reason	Admin
GET	/admin/trust-audit/{provider_id}/	Snapshot history and factor inputs	Admin
POST	/admin/providers/{id}/suspend/	Suspend an account	Admin
POST	/admin/reviews/{id}/hide/	Hide a review with reason	Admin
GET	/admin/job-runs/	Scheduled job health: last-successful-run and overdue flags	Admin

10. Frontend Specification

10.1 Route Map

Route	Purpose	Access
/	Landing: category grid, location picker, value proposition	Public
/services	All categories	Public
/services/:categorySlug	Services within a category	Public
/providers	Search results with filter sidebar	Public
/providers/:id	Provider profile with trust breakdown	Public
/compare	Side-by-side comparison of up to 3 providers	Public
/request-service	Multi-step booking request wizard	Customer
/my-requests	Customer's open and past requests	Customer
/my-bookings	Customer's bookings by state	Customer
/my-bookings/:id	Booking detail, timeline, actions	Customer
/review/:bookingId	Review submission form	Customer
/provider/dashboard	Incoming requests, today's jobs, trust summary	Provider
/provider/bookings	Booking management by state	Provider
/provider/earnings	Earnings ledger and payouts	Provider
/provider/profile	Profile, services, areas, availability	Provider
/provider/trust	Own trust breakdown with improvement guidance	Provider
/login, /register	Authentication	Guest
/admin/*	Verification queue, moderation, trust audit	Admin

10.2 Critical UI Components

- **TrustBadge** — tier label plus score; never rendered without a path to the breakdown.
- **TrustBreakdown** — the six factors as labelled facts with supporting numbers. The signature component of the product.
- **ProviderCard** — search result unit: photo, name, price, TrustBadge, key facts, availability indicator.
- **BookingTimeline** — vertical state history rendered from BookingEvent, so the customer always knows what happened and when.
- **AvailabilityPicker** — calendar plus time-slot grid driven by the availability endpoint.
- **RequestWizard** — service → location → describe → schedule → choose provider → confirm, with state persisted so a refresh does not lose progress.
- **ReviewForm** — overall plus four sub-dimension ratings, with the double-blind notice shown explicitly.

10.3 Key Interaction Requirements

- The booking wizard must be completable on a 360 px-wide screen; mobile is the primary form factor.
- Every state-changing action shows an optimistic UI update with rollback on failure.
- Trust breakdowns are cached per provider for the session; the search list and profile must never disagree on a score.
- All currency is displayed as ■ with thousands separators, never as a bare number.
- Empty states are explicit: 'no providers in this area yet' is a designed screen with a fallback action, not a blank list.

11. Environment and Deployment

No containers are used. Every dependency is installed natively, both locally on Windows and on the production server. This section is written to be followed step by step.

11.1 Local Development Prerequisites

Component	Version	How it is installed on Windows
Python	3.12	python.org installer. Tick <i>Add Python to PATH</i> during setup.
PostgreSQL	18	EDB installer for Windows. Note the postgres superuser password at install time; it is needed once, to create the app database and role. The install directory need not be on PATH — Django connects through psycopg, not the psql CLI.
Node.js	20 LTS or newer	nodejs.org installer, which includes npm. Verified working on 24.x.
Git	any current	git-scm.com.

Nothing else is required. There is no Redis, no broker and no container runtime in Phase 1.

11.2 First-Time Local Setup

```
# 1. Database - run once, in psql as the postgres superuser.
# If psql is not on PATH, call it by full path, e.g.
# "F:\Postgres\bin\psql.exe" -U postgres -h 127.0.0.1
CREATE DATABASE servorabd;
CREATE USER servora WITH PASSWORD '<app password>';
GRANT ALL PRIVILEGES ON DATABASE servorabd TO servora;
ALTER DATABASE servorabd OWNER TO servora;
\c servorabd
-- Required on PostgreSQL 15+: CREATE on public is no longer
-- granted to PUBLIC by default, so migrate would fail without
-- this. No extensions are needed: one that only a superuser
-- can create would make every backup need a superuser to restore.
GRANT ALL ON SCHEMA public TO servora;
ALTER SCHEMA public OWNER TO servora;

# 2. Backend
cd backend
python -m venv venv
venv\Scripts\activate          # PowerShell / cmd
pip install -r requirements.txt
copy .env.example .env         # then edit DB credentials
python manage.py migrate
python manage.py seed_catalogue # categories, services, Dhaka areas
python manage.py createsuperuser
python manage.py runserver      # http://127.0.0.1:8000

# 3. Frontend, in a second terminal
cd frontend
npm install
npm run dev                    # http://127.0.0.1:5173
```

11.3 Scheduled Jobs Locally

The commands in section 8.4 do not run themselves. During development they can simply be invoked by hand when needed, which is usually enough. To exercise the real behaviour, register them with Windows Task Scheduler:

```
# One task per command. Example: expire stale requests, every 15 min.
# Program: F:\...\backend\venv\Scripts\python.exe
# Arguments: manage.py expire_requests
# Start in: F:\...\backend

# Or register from an elevated PowerShell prompt:
schtasks /create /tn "ServorBd-ExpireRequests" /sc minute /mo 15 ^
/tr "F:\path\backend\venv\Scripts\python.exe F:\path\backend\manage.py expire_requests"
```

A `run_scheduled_jobs` convenience command that invokes every sweep once, in order, should also exist — it makes manual testing and the eventual server cron entry trivial.

11.4 Environment Variables

All configuration is read from environment variables via `django-environ`. No secret is ever committed; `.env.example` is committed with placeholder values and documents every key.

```
DEBUG=True
SECRET_KEY=<generated, never reused across environments>
ALLOWED_HOSTS=127.0.0.1,localhost

DATABASE_URL=postgres://servora:<app password>@127.0.0.1:5432/servorabd

EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend
# console backend in dev - prints email to the terminal, so no
# SMTP server is needed and nothing is accidentally sent

OTP_TTL_SECONDS=300
OTP_MAX_SENDS_PER_HOUR=3
JWT_ACCESS_MINUTES=15
JWT_REFRESH_DAYS=14
PLATFORM_COMMISSION_PERCENT=12
TRUST_ALGO_VERSION=v1.0
```

11.5 Production Deployment

Target: a single Ubuntu 24.04 LTS VPS with 2 GB RAM, which ships the same Python 3.12 the project is developed on. Services are installed with `apt` and supervised by `systemd`. Every configuration file and the step-by-step runbook live in `deploy/` in the repository.

Role	Software	Notes
Web server	Nginx	Terminates TLS, serves the built React bundle and public photos under <code>/media</code> , proxies <code>/api</code> to Gunicorn. Sets the real client address that rate limiting relies on.
App server	Gunicorn	3 workers, bound to a Unix socket, managed by a <code>systemd</code> unit with <code>Restart=always</code> .
Database	PostgreSQL 18	Local socket only; not exposed to the network.
TLS	Certbot / Let's Encrypt	Auto-renewing certificate.
Scheduler	cron	Crontab generated by <code>manage.py crontab</code> from the schedule defined in code; every run is recorded and overdue jobs are visible to admins.
Backups	pg_dump + cron	Nightly dump to off-server storage, 30-day retention, and a weekly automatic restore into a scratch database.


```
# Generate the crontab from the job schedule, add backups, install
{ deploy/scripts/manage.sh crontab
  echo '0 2 * * * deploy/scripts/backup.sh'
  echo '30 5 * * 0 deploy/scripts/restore-check.sh'
} | sudo crontab -u servora -

# The generated entries (paths shortened)
*/15 * * * * manage.py expire_requests
5 * * * * manage.py auto_confirm
10 * * * * manage.py reveal_reviews
0 3 * * * manage.py recompute_all_trust
30 3 * * * manage.py reconcile_earnings
*/10 * * * * manage.py purge_otps
0 4 * * * manage.py purge_documents
```

11.6 Deployment Checklist

- `DEBUG=False` and a distinct `SECRET_KEY`, never the development one.
- `ALLOWED_HOSTS` set to the real domain only.
- `python manage.py check --deploy` passes with no warnings.
- `collectstatic` run; Nginx serves static and media directly, never Django.
- Verification-document directory is outside the web root and not served by Nginx under any path.
- Database user is not a superuser and has no rights beyond its own database.
- A restore from backup has been performed successfully at least once.
- Every cron entry has been run manually once and its output checked.

On Docker. Containerising this stack is a worthwhile later improvement — it makes the environment reproducible and simplifies adding Redis when section 8.5's thresholds are reached. It is deliberately not a prerequisite. Nothing in this architecture assumes containers, and nothing needs to change to adopt them.

12. Non-Functional Requirements

12.1 Performance

Metric	Target
Provider search API, p95	< 400 ms
Provider detail API, p95	< 250 ms
Trust recomputation, single provider	< 2 s
Nightly batch, 10,000 providers	< 15 min
Landing page Largest Contentful Paint (4G)	< 2.5 s
API availability	99.5% monthly

12.2 Security

- Verification documents stored in a private bucket, served only via short-lived signed URLs to admin sessions.
- NID numbers stored encrypted at rest; only the last four digits are ever rendered.
- Rate limits: 5 login attempts per 15 min per IP; 3 OTP sends per hour per number; 100 search requests per minute per IP.
- Object-level permissions on every booking, request and review endpoint — a user may only ever read their own party's records.
- Customer address and phone are revealed to the provider only after acceptance, and to no one else.
- All input validated at the serializer boundary; no raw SQL interpolation anywhere.
- HTTPS enforced; HSTS, secure cookies, CSRF protection on session endpoints.
- Trust computation code is server-side only and never exposed as a writable field on any endpoint.

12.3 Reliability and Data

- All multi-write operations wrapped in `transaction.atomic()`.
- Every management command and service function is idempotent, so a double-fired schedule or a retried call is harmless.
- Daily automated database backup with 30-day retention; restore procedure tested before launch.
- `BookingEvent` and `TrustSnapshot` are append-only; deletion is blocked at the service layer.

12.4 Accessibility and Localisation

- WCAG 2.1 AA: keyboard navigable, 4.5:1 contrast, labelled form controls, visible focus states.
- All user-facing strings externalised through i18n from day one, even though only English ships in Phase 1 — retrofitting is far more expensive than preparing.
- Bangla interface in Phase 2. Provider-facing screens are the priority for translation, since providers are the less English-fluent population.
- Currency BDT, timezone Asia/Dhaka, dates rendered in local format.

12.5 Testing

Layer	Approach	Coverage target
Trust engine	Unit tests per factor with table-driven cases including boundaries and zero-data cases. Highest priority suite in the project.	≥ 95%
Booking state machine	Every legal transition and a test that every illegal transition raises.	≥ 95%
Services layer	Unit tests with factory-generated data (factory_boy).	≥ 85%
API	DRF APITestCase per endpoint, including permission denial cases for every role.	≥ 80%
Frontend	Vitest for logic and API normalisers, React Testing Library for the booking wizard and trust components. ESLint runs in CI as a required check.	≥ 60%
E2E	Playwright covering the two golden paths: customer books and reviews; provider accepts and completes.	2 flows

13. Success Metrics

13.1 Product Metrics

Metric	Definition	Target (90 days post-launch)
Request fill rate	Requests accepted by a provider / requests created	≥ 70%
Time to acceptance	Median minutes from request to first acceptance	≤ 30 min
Booking completion rate	Completed / accepted	≥ 85%
Review submission rate	Reviews / completed bookings	≥ 45%
Repeat booking rate	Customers with ≥2 bookings	≥ 25%
Verified provider share	Identity-verified / total active providers	≥ 60%

13.2 Trust System Metrics

These validate that the differentiating feature actually works — they are the most important numbers in the document:

Metric	Why it matters	Target
Trust breakdown view rate	Do customers actually engage with the breakdown, or ignore it like a star rating?	≥ 40% of profile views
Score distribution spread	Standard deviation of trust scores. A tight cluster means the score is not discriminating and the system has failed.	$\sigma \geq 12$
Trust–outcome correlation	Do higher-trust providers actually produce better outcomes? Validates the whole model.	$r \geq 0.5$ vs. completion
Provider score improvement	Share of providers whose score rises after 30 days — proves the score is actionable, not just a label.	≥ 50%
Gaming detection rate	Flagged suspicious patterns investigated per month	Monitored, no target

14. Delivery Plan

Sequenced so that each milestone leaves the system in a demonstrable state. The trust engine is deliberately built early — before search ranking depends on it — because it is the highest-risk and highest-value component.

Milestone	Deliverable	Exit criterion
M1 Foundation	Project scaffold, custom User model, JWT auth, OTP with Postgres-backed expiry, roles, .env config, CI.	A user can register, verify a phone and log in.
M2 Catalogue	Categories, services, locations, admin management, seed data for eight categories and Dhaka areas.	Catalogue browsable via API and admin.
M3 Providers	Provider profile, offered services, service areas, availability, verification upload and admin review.	A provider can build a complete, verifiable profile.
M4 Trust engine	All six factors, composition, penalties, tiers, snapshots, the <code>recompute_all_trust</code> command, full unit test suite.	<code>docs/verify_trust_math.py</code> passes: both worked examples in 7.4 reproduce exactly.
M5 Discovery	Search, filters, sort by trust, provider detail, trust breakdown endpoint.	Search returns trust-ranked results under 400 ms.
M6 Booking	Request creation, accept/decline, full state machine, event log, cancellation, expiry and auto-confirm jobs.	Full lifecycle passes E2E test.
M7 Reviews	Double-blind review submission, sub-ratings, provider reply, moderation, trust feedback loop.	A review measurably moves the provider's score.
M8 Money	Cash completion recording, commission accrual, earnings dashboard.	Provider earnings reconcile against bookings.
M9 Frontend	All customer and provider routes, mobile-first, notification centre.	Both golden paths complete in the browser on a phone.
M10 Harden	Rate limiting, security review, performance tuning, accessibility pass, seed/demo data, README with the trust algorithm documented, VPS deploy with all cron entries registered and verified.	Deployed, load-tested, documented; every scheduled command has run successfully in production.

14.1 Risks

Risk	Impact	Mitigation
Cold-start: no providers, so no customers	High	Launch in one or two Dhaka thanas only. Recruit 20–30 providers per category manually before opening to customers.
Trust score is unfair to new providers	High	F1 lets a new provider reach ~35 points on day one through verification alone; the 'New' tier is presented honestly rather than as a low score; a new-provider search slot is reserved on result pages.
Providers transact off-platform after first contact	High	Phone and address revealed only after acceptance; reviews and trust accrue only on-platform, so leaving costs the provider their reputation asset.
Trust weights turn out to be wrong	Medium	Weights are configuration, snapshots record <code>algo_version</code> , and every score is reproducible — so weights can be retuned and the effect measured against historical data.
Verification review becomes an ops bottleneck	Medium	Queue with SLA target of 24h; automated pre-checks for image quality and NID format before a human sees it.

Risk	Impact	Mitigation
Scope creep beyond Phase 1	Medium	Section 4.4 records descoped items explicitly; anything not in section 4.1 requires a written scope change.
Synchronous work outgrows the request cycle	Medium	Accepted deliberately to avoid a broker in Phase 1. Section 8.5 defines the thresholds that trigger the move and the three service-layer rules that keep the migration to one line per call site. Booking-completion latency is monitored from M6 so the threshold is observed, not guessed.
A missed cron entry silently stops a sweep	Medium	Each command logs a run record; an admin dashboard panel shows last-successful-run per job, so a stalled scheduler is visible rather than silent.

15. Open Questions

#	Question	Owner	Needed by
Q1	Is 12% the right commission rate, and should it vary by category? High-ticket AC work may not bear the same rate as a 300-taka cleaning job.	Product	M8
Q2	Should cancellations by the customer affect the <i>customer's</i> standing? A customer reliability score is symmetric and would reduce provider-side wasted travel.	Product	Phase 2
Q3	What is the minimum job value below which F2 volume contribution is reduced, to prevent farming?	Product	M4
Q4	Should the platform enforce price ceilings against the suggested band, or only flag outliers to customers?	Product	M3
Q5	For visit-fee-plus-quote services, is the quote negotiated in-app in Phase 2 or handled verbally?	Product	Phase 2
Q6	Does the provider see their own factor breakdown with improvement guidance? Recommended yes — it converts the score from a judgement into a tool — but it also teaches the system to bad actors.	Product	M4

15.1 Recommendations on the Open Questions

Q6 in particular should be answered yes. A score a provider cannot understand or influence is experienced as arbitrary, and arbitrary scoring drives good providers off the platform. The information an honest provider uses to improve is the same information a bad actor would use to game — but the anti-gaming measures in section 7.5 are designed to hold under full transparency, which is precisely why they are behaviour-based rather than secret.

15.2 What Was Removed From the Original Concept

For the record, and to explain the deltas from the initial outline:

- **Report model** moved to Phase 2. Abuse reporting matters, but with no users there is nothing to report; it would be dead code at launch.
- **Payment gateway** moved to Phase 2. Integrating bKash or SSLCommerz requires a merchant account and a compliance process that would block the entire build behind an external dependency. Cash settlement is recorded on-platform from day one, so the data model and commission logic are exercised and the gateway drops in without schema change.
- **Separate Availability endpoint per provider** kept, but the model extended with dated exceptions — the original weekly-only design could not express 'on leave next Tuesday', which providers need constantly.

And what was added: the double-blind review mechanism, the BookingEvent audit log, TrustSnapshot with algorithm versioning, request expiry and booking auto-confirmation, the dispute state, the anti-gaming design, and the trust-system success metrics that make it possible to tell whether the signature feature actually worked.

Closing note

The technically interesting part of this project is not the CRUD. It is section 7. A booking marketplace is a well-understood problem; a transparent, gameable-resistant, statistically sound trust model with a full audit trail is not. Build the trust engine first, test it hardest, and document it in the README — that is what makes this project worth showing.

